

CrowdArb — Problems Encountered & Solutions

Aris Rault

July 2026

A running log of real problems hit during the build and how we solved them. Useful for interviews, documentation, and remembering what actually happened.

PROBLEM 1 — POLYMARKET API RETURNED ZERO FED CONTRACTS ON FIRST TRY

Layer: 0 — Hello Data.

What happened: The first version of `layer0_data.py` searched Polymarket’s API for Fed rate contracts and returned nothing.

Why: The API only returns 100 markets per request (one “page”). Fed rate contracts happened to sit in the second batch (offset 100–200). The first script only asked for the first page.

Solution: Added pagination — the script now loops through 4 pages of 100 markets each (offsets 0, 100, 200, 300), scanning 400 markets total. Fed contracts appeared at offset 100.

Lesson: Always check API pagination. Never assume one request returns everything.

PROBLEM 2 — CME FEDWATCH API BLOCKED DIRECT ACCESS (403 ERROR)

Layer: 0 — Hello Data.

What happened: Tried to pull Fed rate cut probabilities directly from CME’s FedWatch API endpoint. Got a 403 (Forbidden) response — CME blocks programmatic access to their data.

Solution: Replicated FedWatch’s own formula manually using two free data sources from Yahoo Finance: `^IRX` (13-week T-bill rate as a proxy for the current Fed funds rate) and `ZQM26.CBT` (30-day Fed funds futures contract for the June 2026 meeting month).

The formula: a Fed funds futures contract prices the *average* overnight rate across the whole calendar month. The June FOMC meeting falls on day 18, splitting the month into 17 pre-meeting days and 13 post-meeting days. Solving for the implied post-meeting rate:

```
r_after = (r_implied * 30 - 17 * r_current) / 13
P(cut) = (r_current - r_after) / 0.25
```

The 0.25 is one standard Fed move (25 basis points). This is the exact algebra CME uses on their public FedWatch page.

Result: 11.5% implied probability of a cut at the June 18, 2026 meeting.

Lesson: When an API blocks you, find the underlying data and replicate the formula yourself. This is actually more educational — you understand the math instead of just reading a number off a webpage.

PROBLEM 3 — `layer0_data.py` ACCIDENTALLY EMPTIED DURING COMMENT CLEANUP

Layer: 0 — Hello Data.

What happened: During a cleanup of verbose comments in `layer0_data.py`, the file was accidentally emptied entirely — 95 lines deleted, 0 added — and the empty file was committed and pushed to GitHub.

Solution: Restored the file by rewriting the working Polymarket fetching logic from scratch with clean, short comments, then re-committed.

Lesson: When asking for cosmetic changes (comment cleanup, formatting), always verify the file still has content before committing. Run `git diff` before `git push` to see exactly what changed.

PROBLEM 5 — BACKTEST RESULTS CHANGED ON EVERY RUN DESPITE A “FIXED SEED”

Layer: 2 and 3 — Avellaneda-Stoikov backtest and cross-market calibration.

What happened: The README claimed `seed=42` made results reproducible, but running the same script twice gave different P&L variance and Brier scores each time.

Why: The backtest’s starting probability (`p0`) was being set by calling `fetch_polymarket()` — pulling today’s live price. Since Polymarket’s price drifts day to day, the random walk always started from a different point, even though the random number generator itself might have been seeded correctly. The “fixed seed” only fixed the randomness, not the starting condition.

Solution: Separated the two concerns. Live data fetching stayed in Layer 0’s “Hello World” scripts for the live demo. The backtests now use a hardcoded constant (`BACKTEST_P0 = 0.72`) completely disconnected from live data, so every run starts from the same point and produces identical results.

Lesson: “Reproducible” means every input is fixed, not just the random seed. A backtest that pulls live data for any part of its setup is not reproducible, no matter how carefully the randomness is seeded. Always verify reproducibility by actually running the script twice and comparing outputs — don’t trust a `seed=X` comment without checking.

PROBLEM 6 — FED PROBABILITY SHOWED AN IMPOSSIBLE 100% NEAR MONTH-END MEETINGS

Layer: 0 — CME FedWatch professional probability.

What happened: The scanner showed the Fed professional probability as 100% (clamped down from a value the formula computed as 1.74). A 174% probability is mathematically impossible, and the 100% it got clamped to was the one visibly-wrong number on the dashboard.

Why: The professional probability is extracted from a futures contract that prices the average overnight rate across a calendar month. The post-meeting rate is found by removing the pre-meeting days from that average. When the FOMC meeting falls near the end of the month (July 30 sits on day 30 of 31), only 2 days of the contract are post-meeting. Extracting a 2-day signal out of a 31-day average amplifies any small price difference by a factor of $31/2 \approx 15.5\times$. A normal 2.8 basis-point discount got amplified into an implied 43 basis points, which reads as 1.74 rate cuts.

Solution: When fewer than 5 days remain after the meeting in the current-month contract, switch to the next month’s futures contract and read its implied rate directly. The next month is entirely post-meeting (31 clean days), so there’s no tiny denominator and no amplification. This matches CME’s own published FedWatch methodology, which switches to the next-month contract for late-month meetings for exactly this reason.

Result: The Fed professional probability now reads a sensible 0% for the July 30 meeting — a genuine market signal (August futures are priced above the current rate, implying no cut expected) rather than a numerical artifact. This revealed a clean ~20-point gap between Polymarket’s 20% and the professional 0%, the clearest crowd-vs-professional divergence in the project.

Lesson: Dividing by a small number amplifies noise. When a formula extracts a signal from a shrinking window, watch for instability as that window approaches zero — and check how the established industry handles the same edge case rather than inventing a workaround.

DESIGN DECISION 1 — SCANNER BLACK-SCHOLES SANITY-FILTER THRESHOLD

Layer: Phase G — real-time market auto-discovery.

The decision: `discover_markets()` filters out any price-level contract whose Black-Scholes implied probability falls outside a sanity band before showing it in the scanner. The band was set to 0.5%–98% (loosened from an initial 2%–98%).

Why a filter exists at all: Without it, the scanner surfaced absurd contracts like “Bitcoin to \$1M before GTA VI” (a 16.7x move in 34 days, ~0% probability), which broke the blender (it pushed 100% weight to one source) and the market-maker spread (fair value near 0). The filter keeps the scanner on contracts where the crowd-vs-professional comparison is actually meaningful.

Why the band was loosened to 0.5%: At a 2% lower bound, only ~4 contracts survived, because most higher-strike crypto contracts (BTC \$130k+, etc.) are genuinely near-impossible under Black-Scholes and got dropped. Lowering the floor to 0.5% lets more strikes through, giving a fuller scanner table for presentation, while the 98% upper bound still removes near-certain (no-disagreement) contracts.

The honest tradeoff to mention when presenting: A wider band shows more rows but some have very low professional probabilities where the two sources largely agree, so the gap is less interesting. A tighter band shows fewer but more meaningful rows. The 0.5% floor is a presentation choice favouring a fuller table; the most decision-relevant rows are still the ones with the largest gaps, which sort to the top regardless of the threshold.

To note when reporting/presenting: the set of markets shown is a function of this threshold — it is a tunable design parameter, not a fixed property of the data. State this explicitly so the scanner’s contents are understood as a filtered view, not the entirety of what exists.

More problems will be added as the project progresses.